BAHRIA UNIVERSITY, (Karachi Campus)

Department of Software Engineering

Assignment 4 - Spring 2025

COURSE TITLE:	DevOps	COURSE CODE:	SEN-429
Class:	**BSE-8 A & B**	Shift:	**Morning**
Course Instructor:	Sohaib Ur Rehman	Time Allowed:	**1 Week**
Submission Date:	**28th May 2025**	Max. Marks:	**5 Marks**

Name: Muhammad Shoaib Akhter Qadri Enrollment No: 02-131212-009

Assignment Title: Securing S3 Static Website Hosting with Custom Bucket Policies [CLO5: 5 Marks]

This assignment simulates a scenario where a public static website is hosted on Amazon S3. You must ensure the site is accessible while enforcing **strict security and compliance controls** using custom S3 bucket policies generated via the **AWS Policy Generator**

Tasks:

1. **Static Website Deployment**

   - Create an **S3 bucket** with a unique name.
   - Enable **Static Website Hosting** for the bucket.
   - Upload basic HTML files (`index.html` and `error.html`).
   - Ensure the website is publicly accessible and share the URL.
   - Provide screenshots of:
     - Bucket configuration
     - File upload confirmation
     - Working website in the browser

2. **Bucket Policy Creation Using Policy Generator**

   - Use the **AWS Policy Generator** to create a **custom S3 bucket policy**.
   - The policy must explicitly **Deny** the following actions for all principals (`*`):
     - `s3:DeleteObject`
     - `s3:PutObject`
     - `s3:DeleteBucket`
   - Attach the policy to the bucket and test that these actions are denied.
   - Submit:

     - A copy of the JSON policy

- Screenshots showing the policy applied
- Attempt the denied actions and provide evidence (screenshots or error messages).
- Explain how this policy aligns with **security best practices** in a DevOps or cloud environment

Submission Instructions:

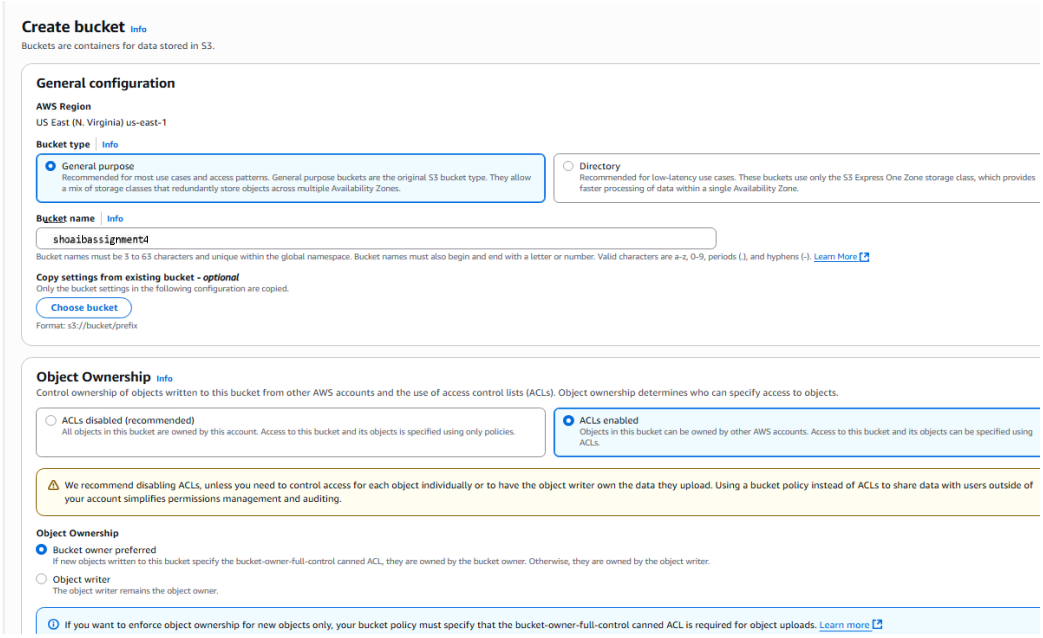
- Submit a single **PDF** document containing:
  - Website URL
  - Required screenshots
  - JSON policy
  - Explanations

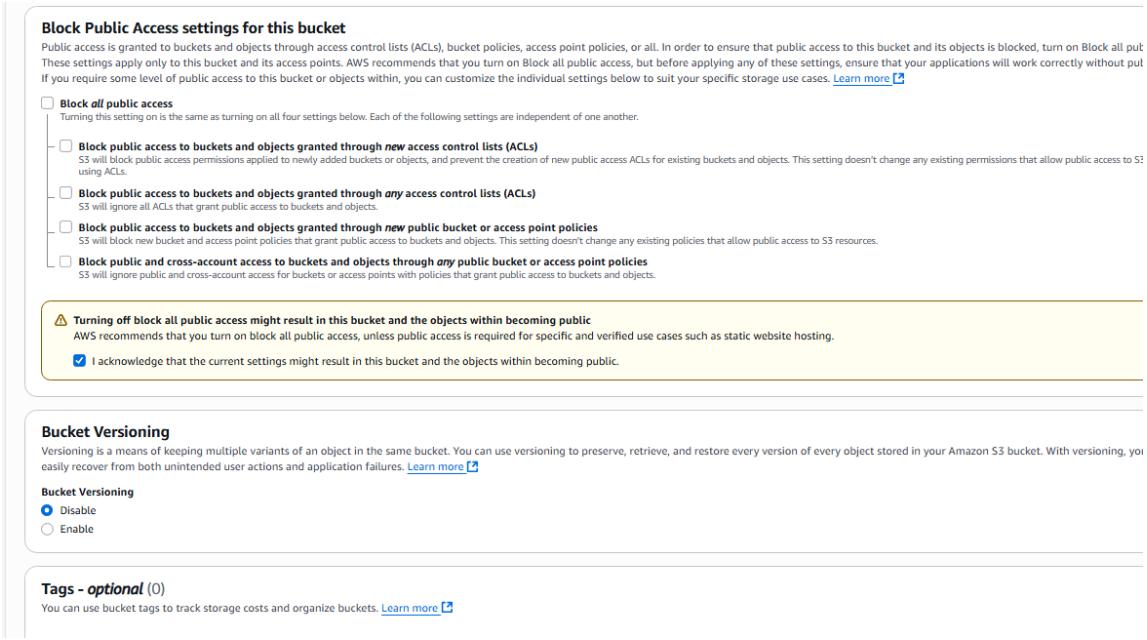
Website URL:

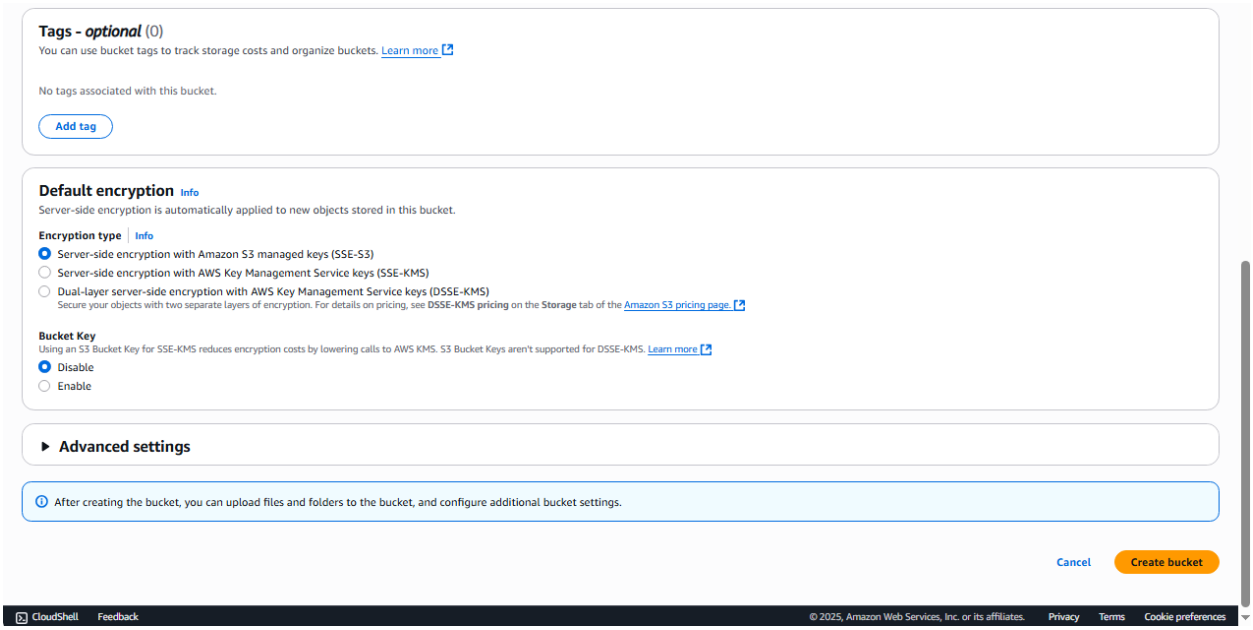
Required screenshots with Explanations:

1. **First we will create a bucket and give**

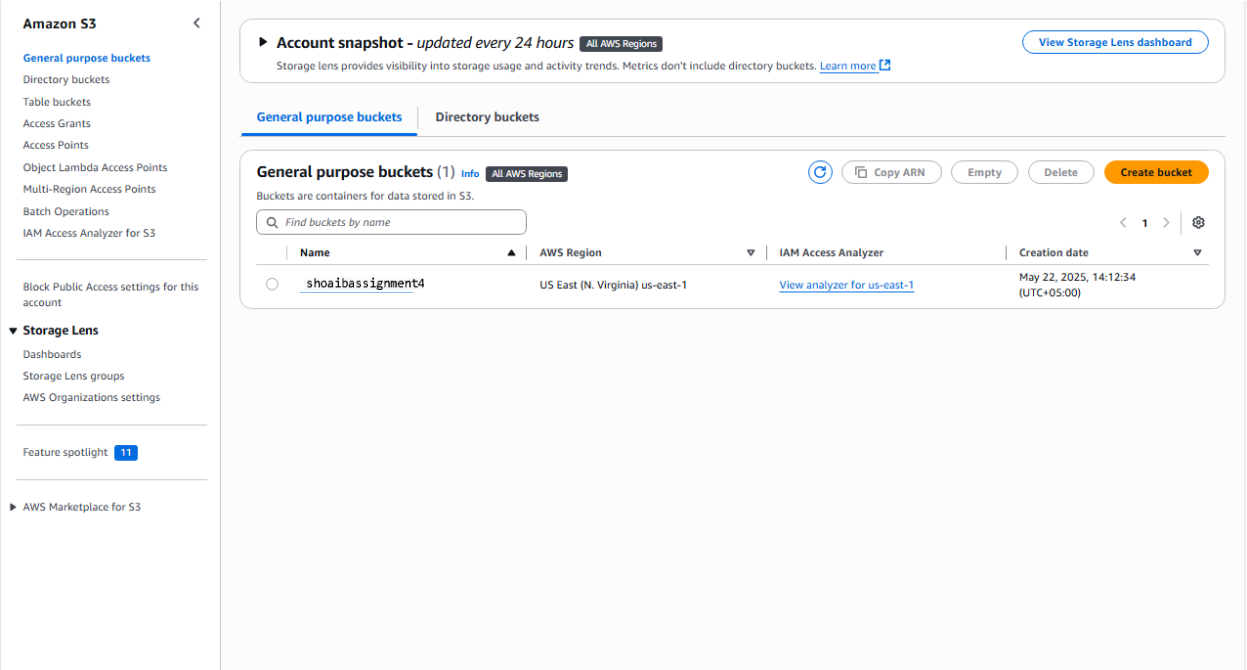
Bucket Name = shoaibassignment4
Bucket Type = General Purpose
Object Ownership = ACLs enabled
Object Ownership= Bucket owner preferred
Bucket Vision=Disabled
BucketKey= disabled



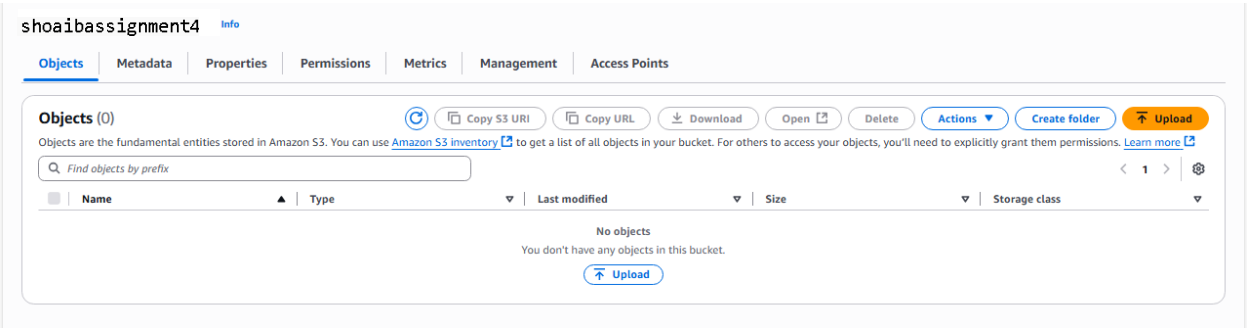

Block Public Access settings for this bucket
Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to this bucket and its objects is blocked, turn on Block all pub
These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without pul
If you require some level of public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use cases. Learn more
Block all public access
Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.
Block public access to buckets and objects granted through new access control lists (ACLs)
S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3
using ACLs.
Block public access to buckets and objects granted through any access control lists (ACLs)
S3 will ignore all ACLs that grant public access to buckets and objects.
Block public access to buckets and objects granted through new public bucket or access point policies
S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.
Block public and cross-account access to buckets and objects through any public bucket or access point policies
S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.
Turning off block all public access might result in this bucket and the objects within becoming public
AWS recommends that you turn on block all public access, unless public access is required for specific and verified use cases such as static website hosting.
I acknowledge that the current settings might result in this bucket and the objects within becoming public.
Bucket Versioning
Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, yo
easily recover from both unintended user actions and application failures. Learn more
Bucket Versioning
Disable
Enable
Tags - optional (0)
You can use bucket tags to track storage costs and organize buckets. Learn more


Tags - optional (0)
You can use bucket tags to track storage costs and organize buckets. Learn more
No tags associated with this bucket.
Add tag
Default encryption Info
Server-side encryption is automatically applied to new objects stored in this bucket.
Encryption type Info
Server-side encryption with Amazon S3 managed keys (SSE-S3)
Server-side encryption with AWS Key Management Service keys (SSE-KMS)
Dual-layer server-side encryption with AWS Key Management Service keys (DSSE-KMS)
Secure your objects with two separate layers of encryption. For details on pricing, see DSSE-KMS pricing on the Storage tab of the Amazon S3 pricing page.
Bucket Key
Using an S3 Bucket Key for SSE-KMS reduces encryption costs by lowering calls to AWS KMS. S3 Bucket Keys aren't supported for DSSE-KMS. Learn more
Disable
Enable
Advanced settings
After creating the bucket, you can upload files and folders to the bucket, and configure additional bucket settings.
Cancel
Create bucket
CloudShell
Feedback
© 2025, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

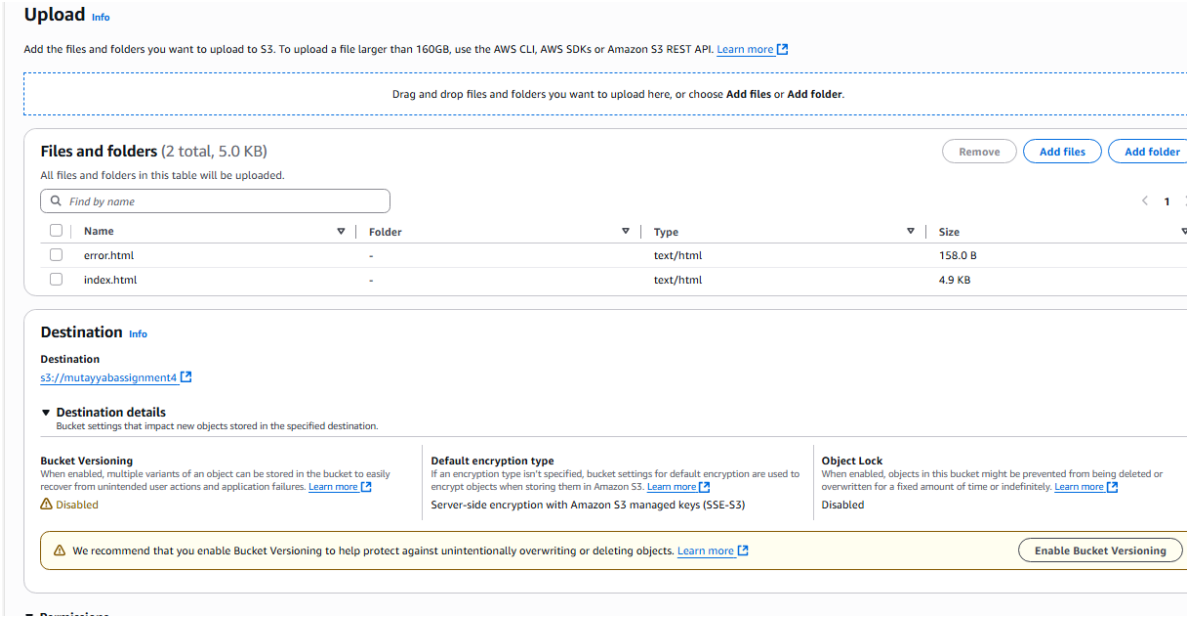
Now Bucket has been created successfully



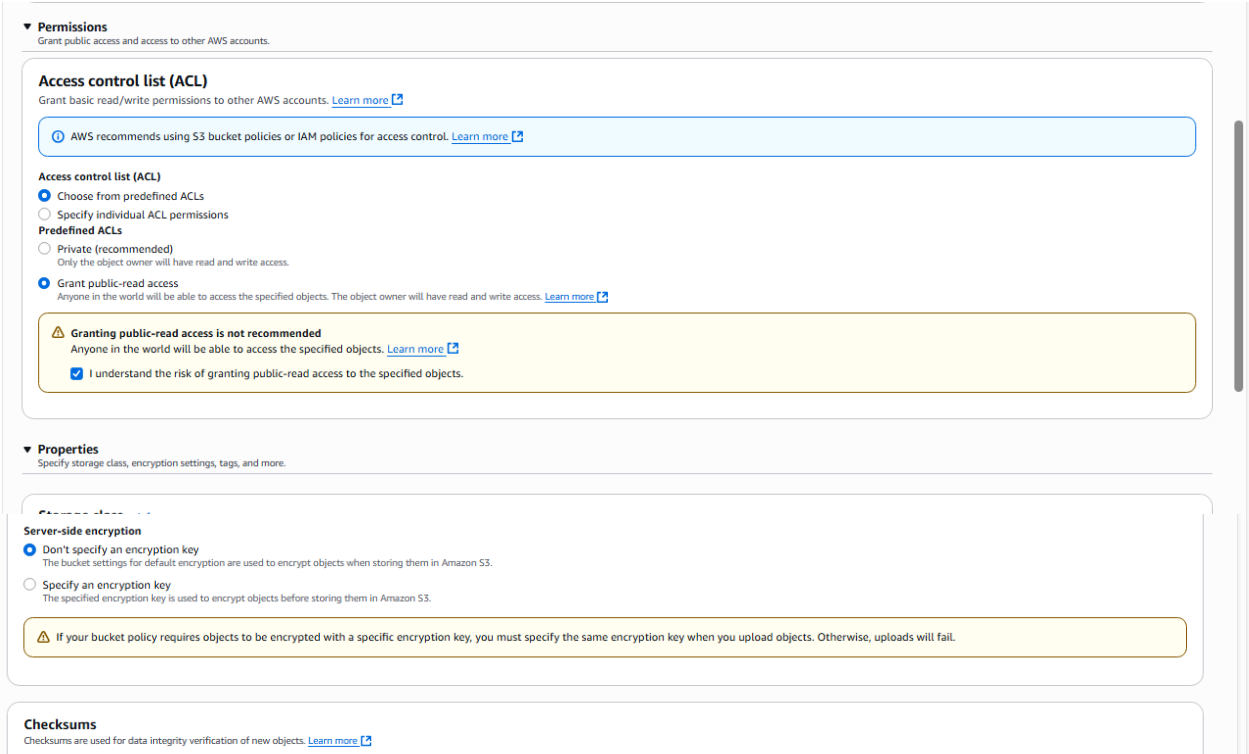
2. Now Upload index.html and error.html file



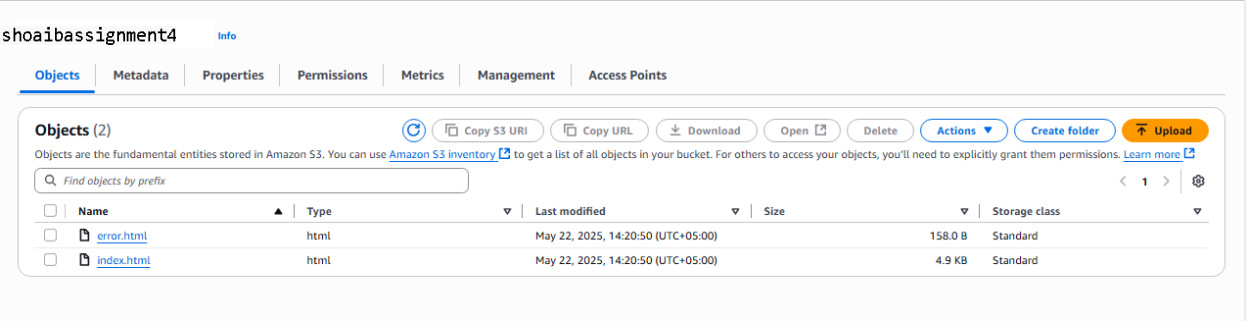
3. Now Select the Files using the Add File Button



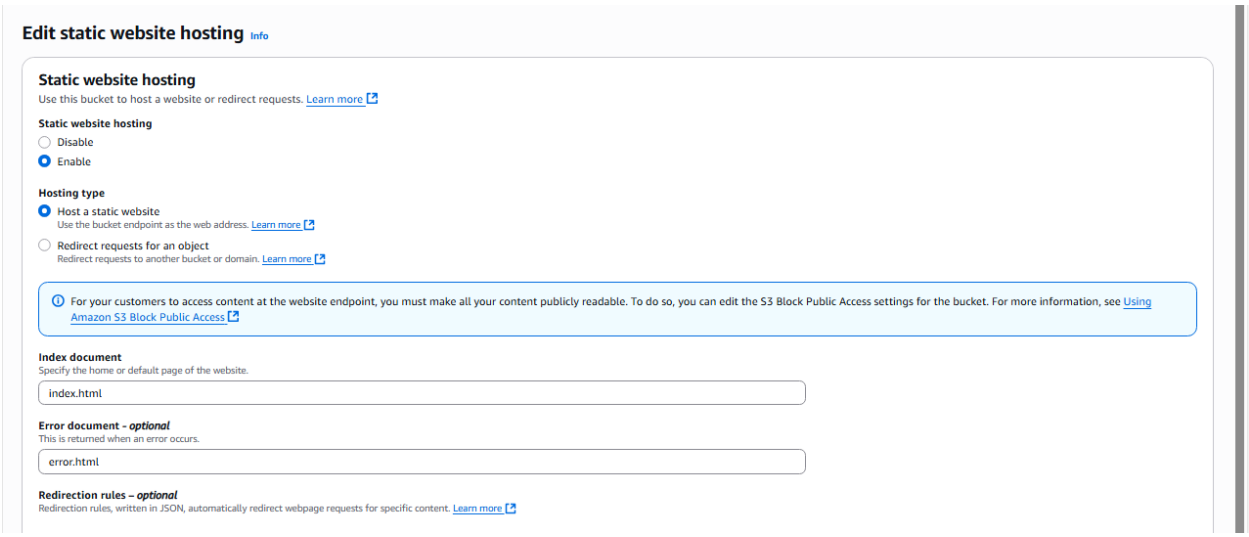
4. Select the Grant-public-read access under Access Control List (ACL) Permissions.

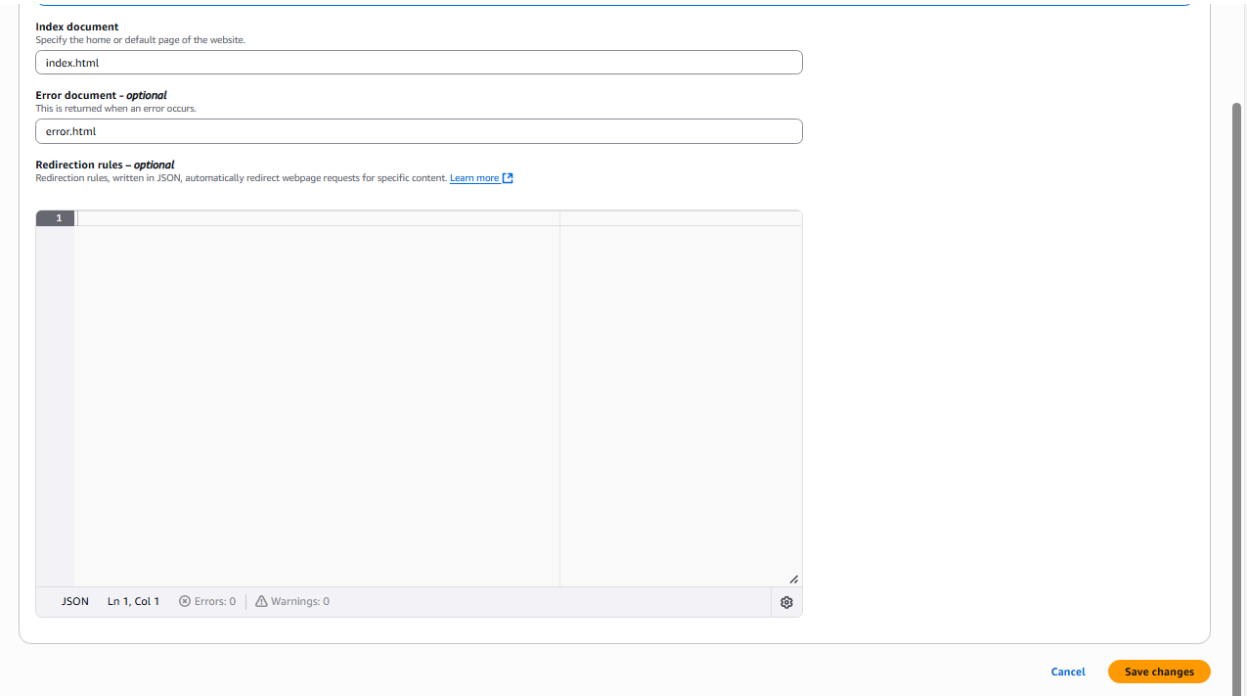


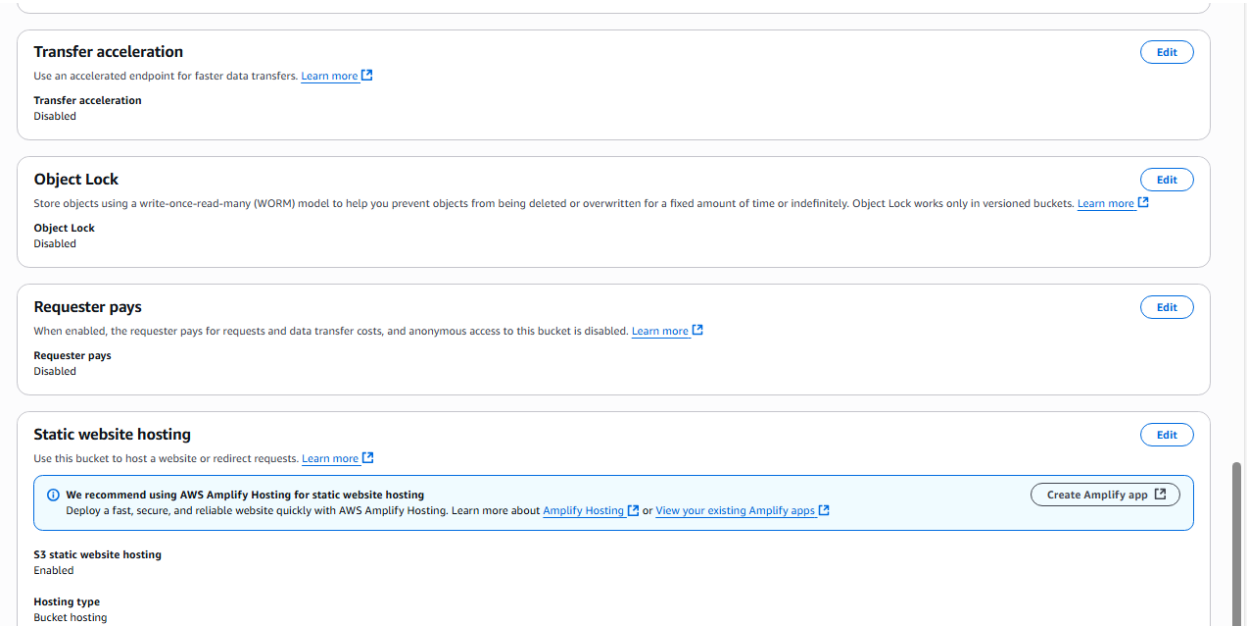
5. **Now files rre uploaded**



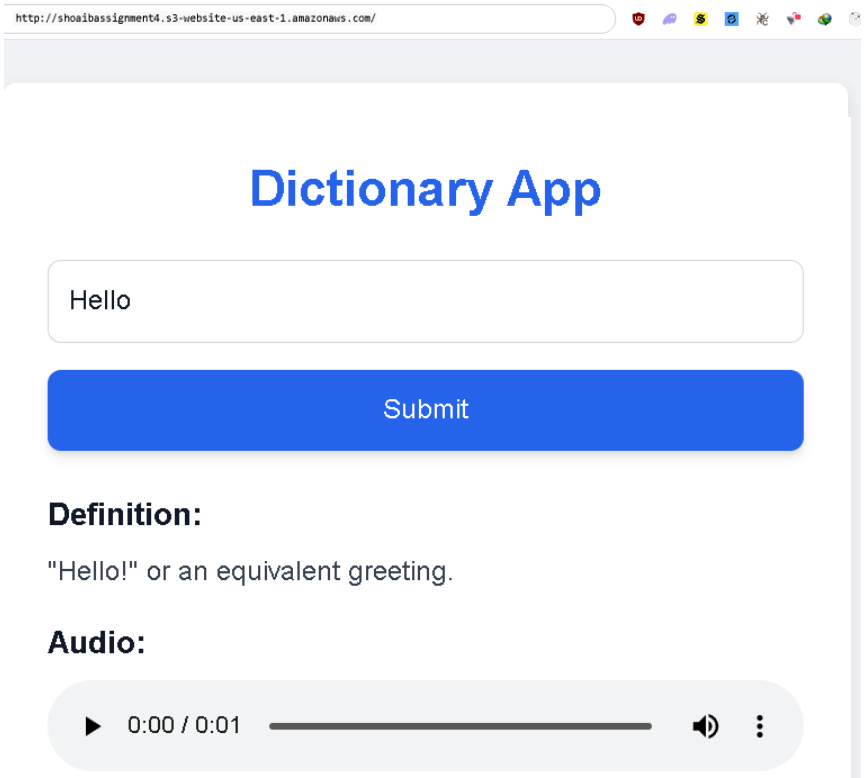
6. **Now Enable the Static Website Hosting Option under the properties tab**





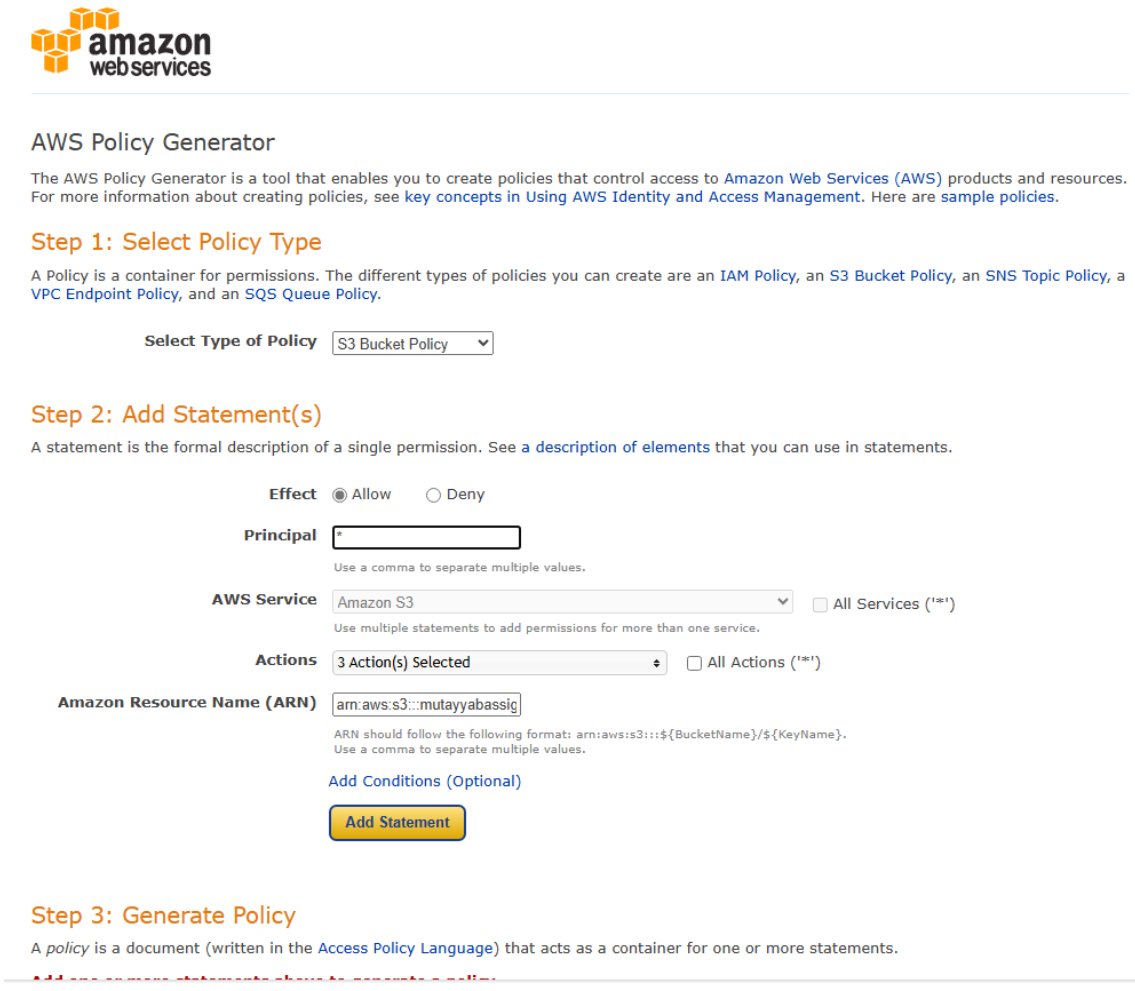


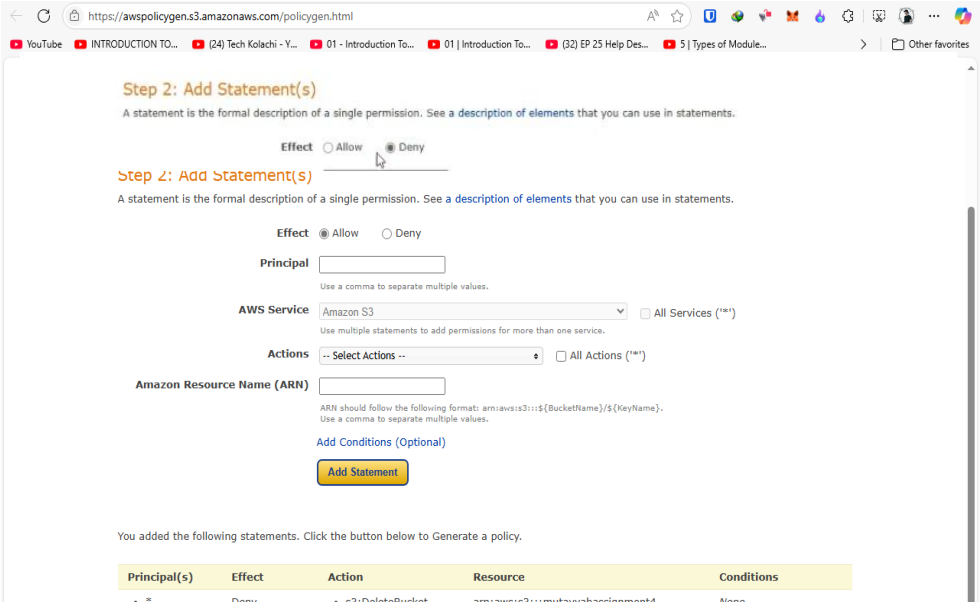
This is the URL where you can access your website.
http://shoaibassignment4.s3-website-us-east-1.amazonaws.com/



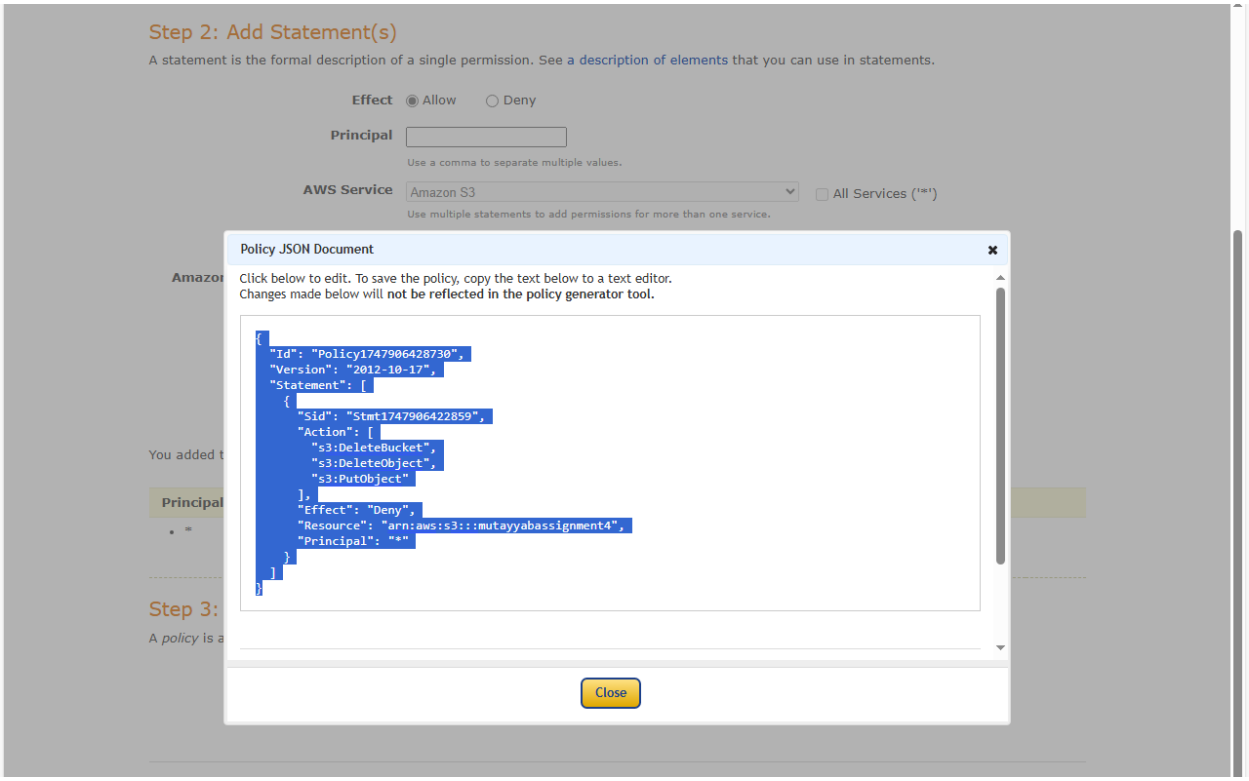
Bucket Policy Creation Using Policy Generator

Now we will create a custom S3 bucket policy using AWS Policy Generator

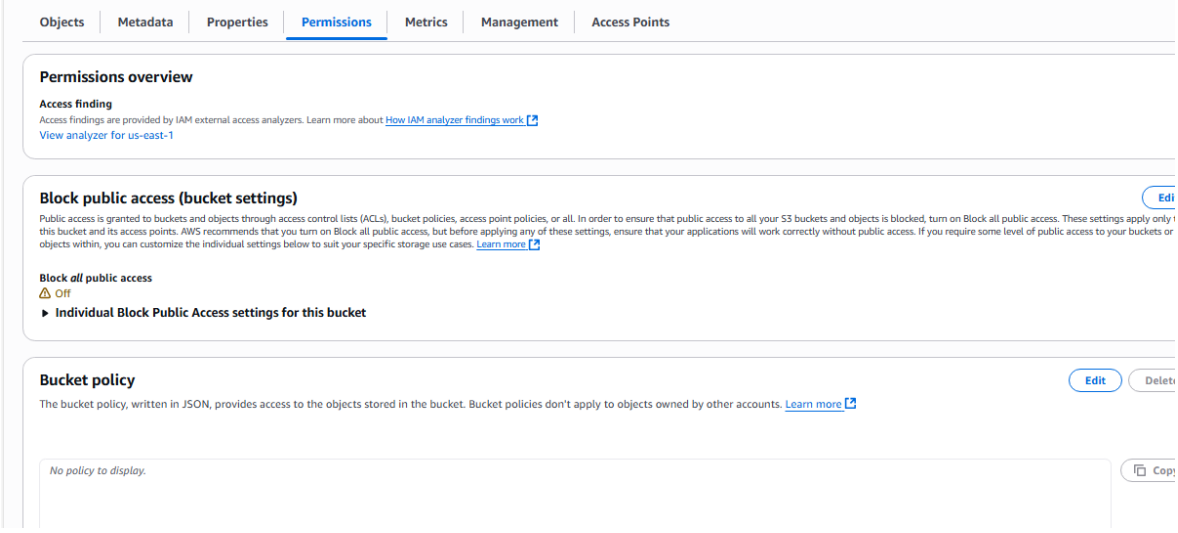




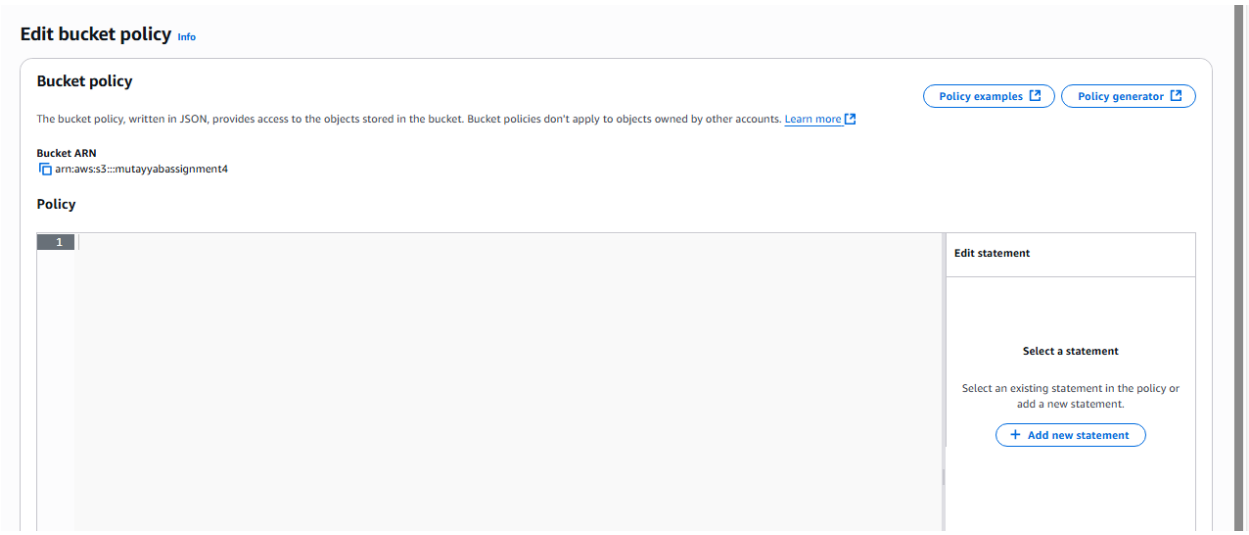
Here is the Policy JSON Document



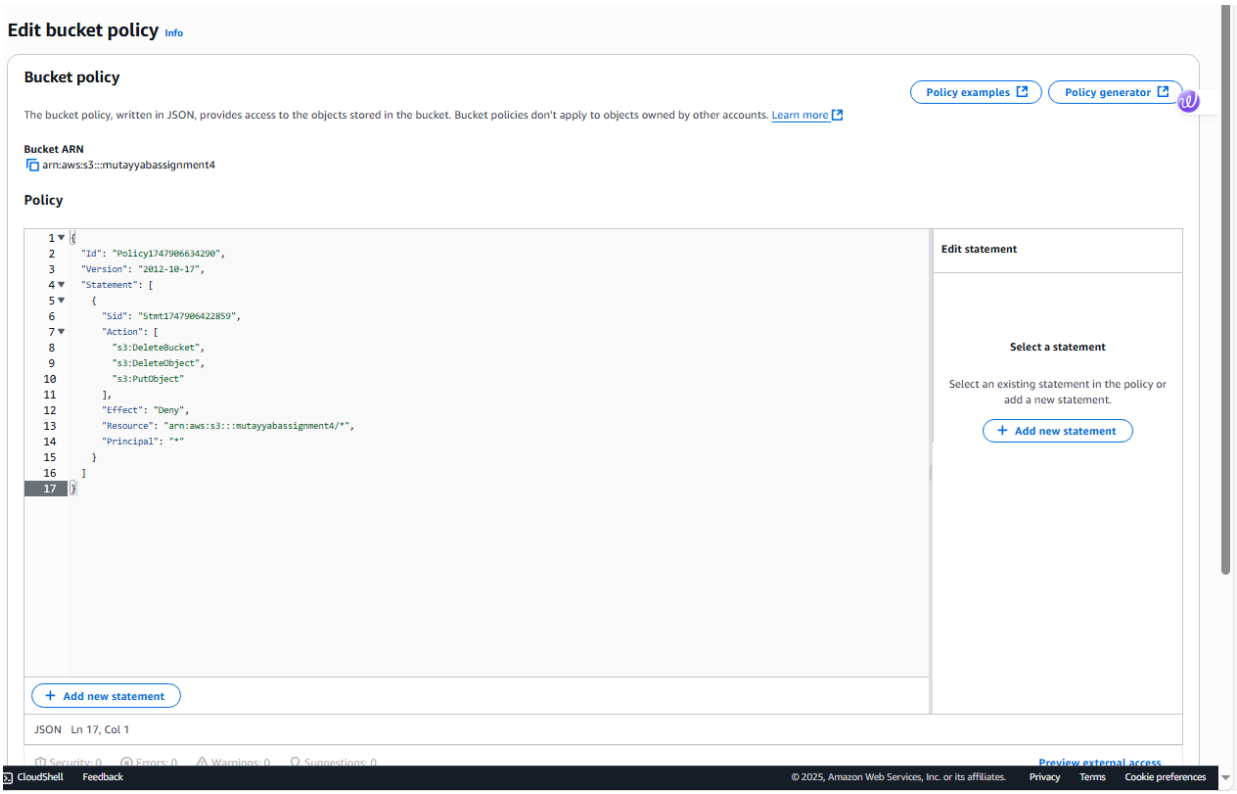
7. **Now we will Add the policy which we created using the AWS Policy Generator under the Permission > Bucket policy > Edit**



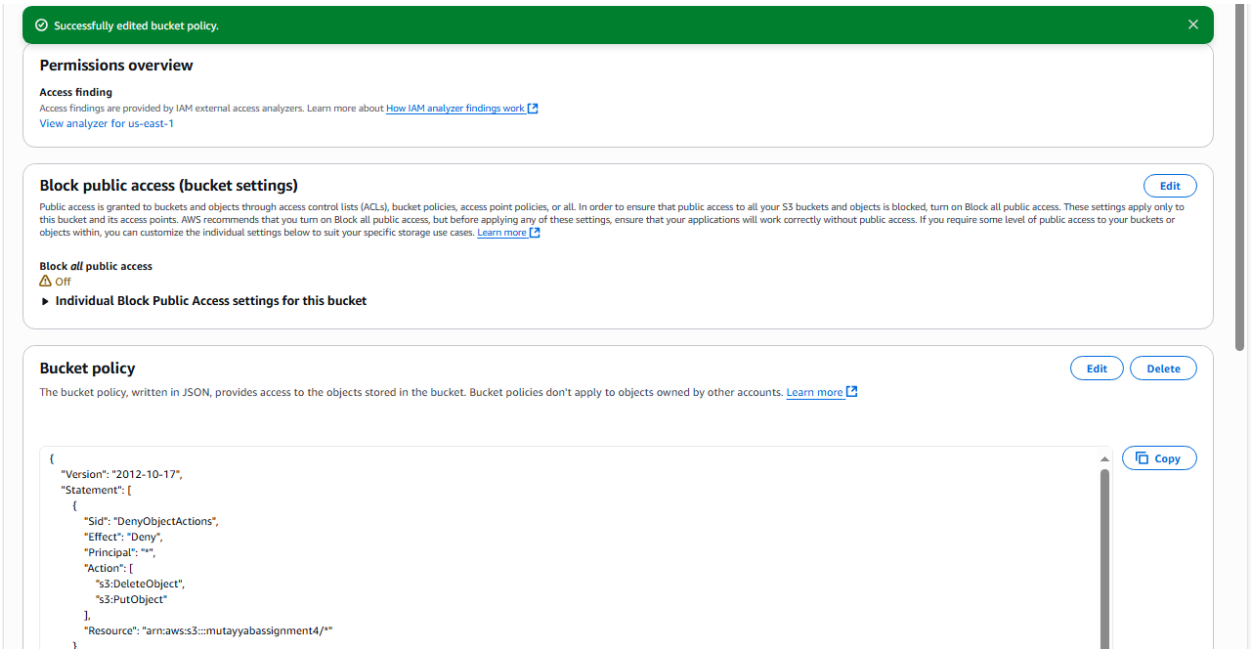
Here is the Edit Bucket Policy



Now we are pasting policy here



Now Bucket Policy has been successfully



The Policy We created using the AWS Policy Generator is not working so I will rewrite the policy.
The problem is that you're trying to apply all three actions (DeleteBucket, DeleteObject, and PutObject) to the same resource pattern (arn:aws:s3:::mutayyabassignment4/*), but DeleteBucket is a bucket-level operation that can't be applied to objects (/*)

```
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Sid": "DenyObjectActions",
   "Effect": "Deny",
   "Principal": "*",
   "Action": [
    "s3:DeleteObject",
    "s3:PutObject"
   ],
   "Resource": "arn:aws:s3:::mutayyabassignment4/*"
  },
  {
   "Sid": "DenyBucketDeletion",
   "Effect": "Deny",
   "Principal": "*",
   "Action": "s3:DeleteBucket",
```

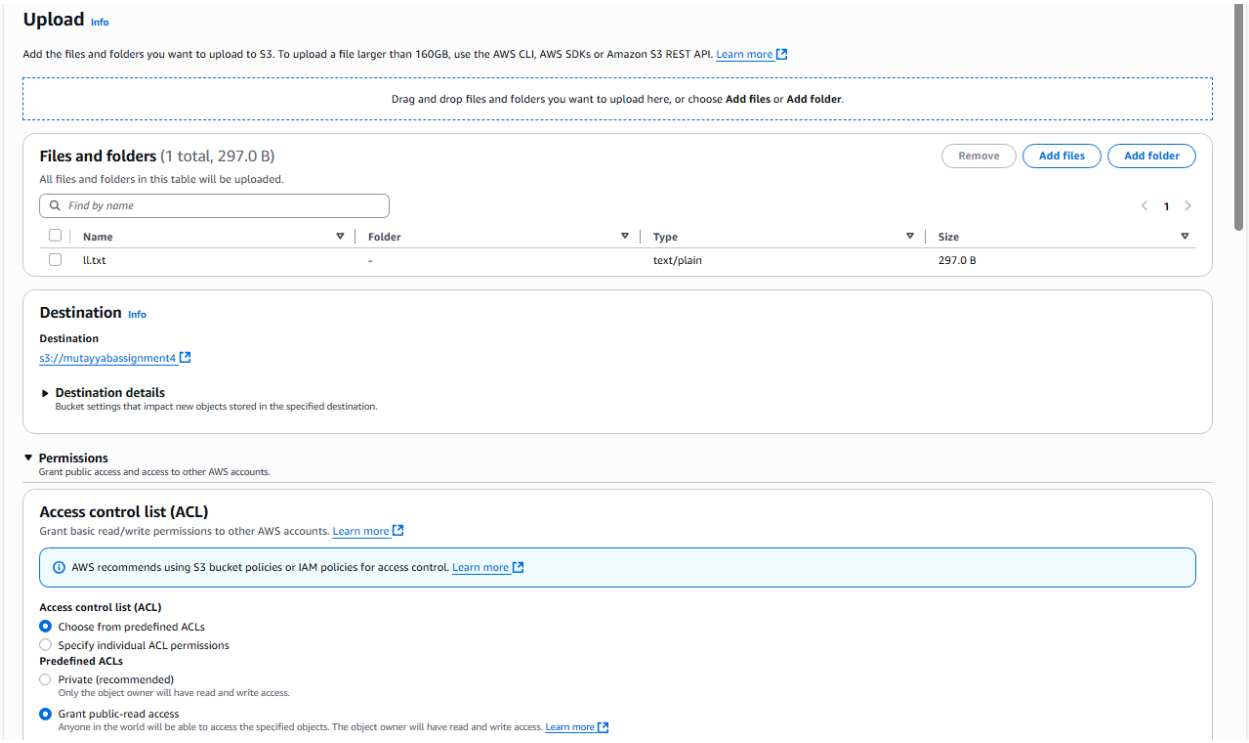
```
    "Resource": "arn:aws:s3:::mutayyabassignment4"
   }
  ]
}
```

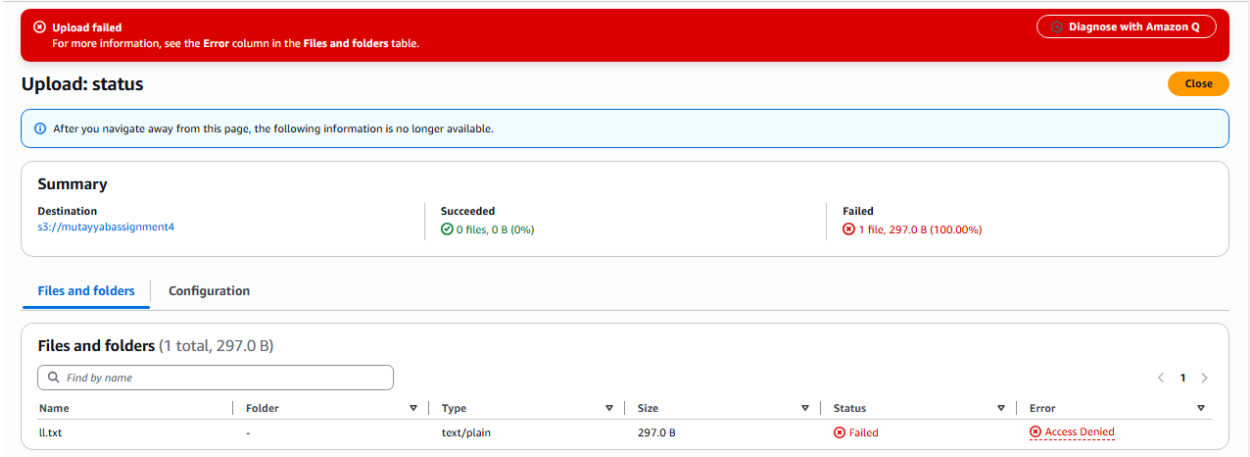
Key Fixes:

1. **Separated the statements**:
   - One for object-level actions (DeleteObject, PutObject) that need /*
   - One for bucket-level action (DeleteBucket) that must NOT have /*
2. **Removed the "Id" field** (not required for bucket policies)
3. **Proper resource ARNs**:
   - For objects: arn:aws:s3:::mutayyabassignment4/*
   - For bucket: arn:aws:s3:::mutayyabassignment4

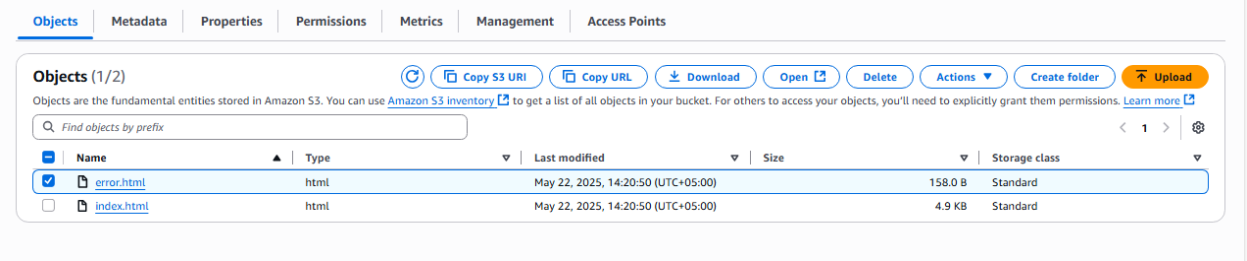
Now we Will Check our policy is working or not

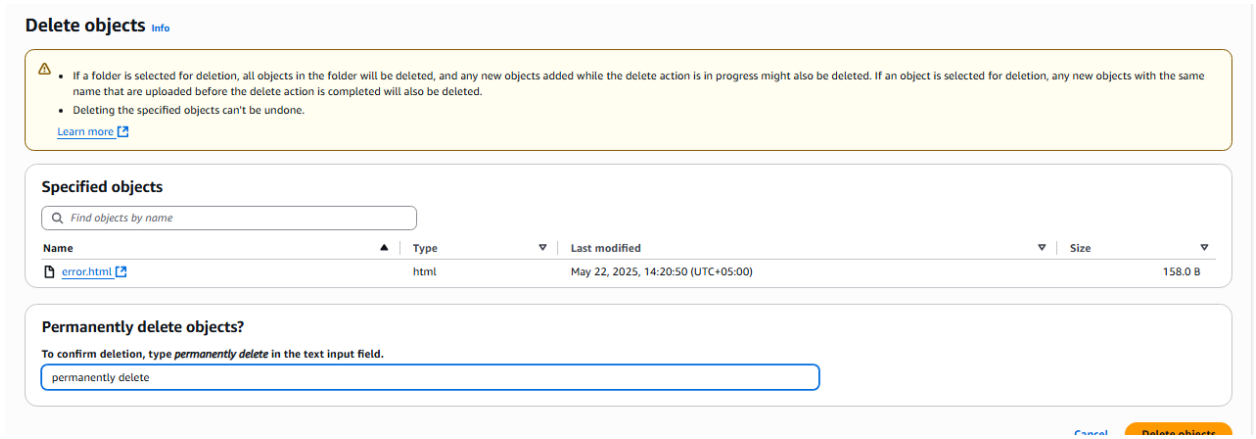
Uploading a file to check the **PutObject** Functionality under the Upload File Option.

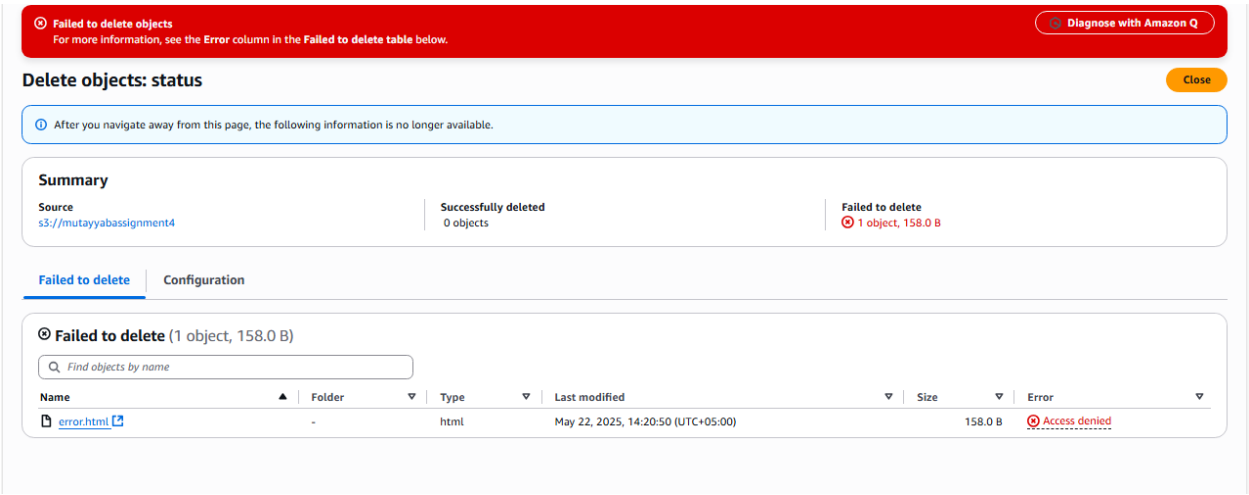




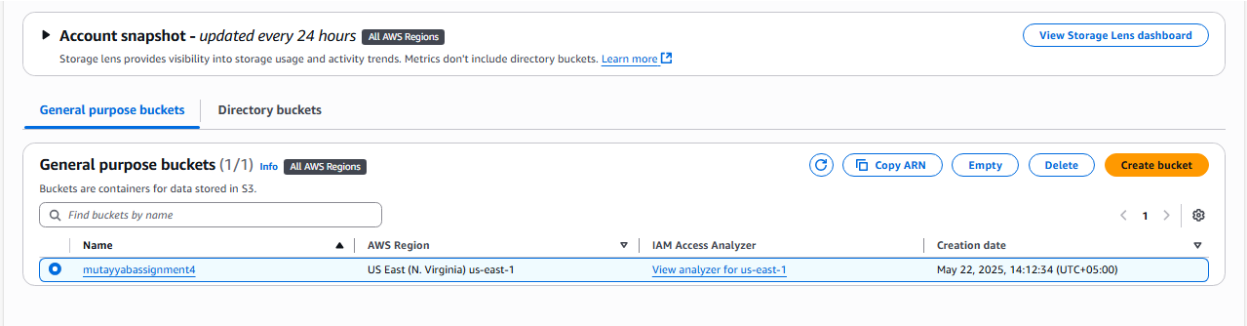
Deleting a File to check the **DeleteObject** Functionality under the Delete File Option.



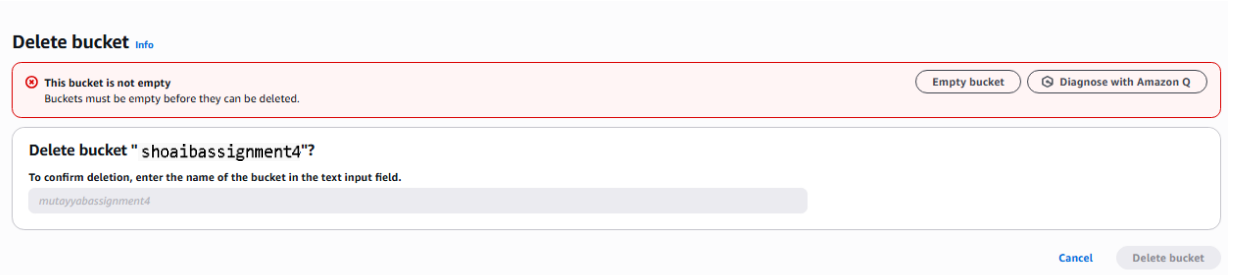




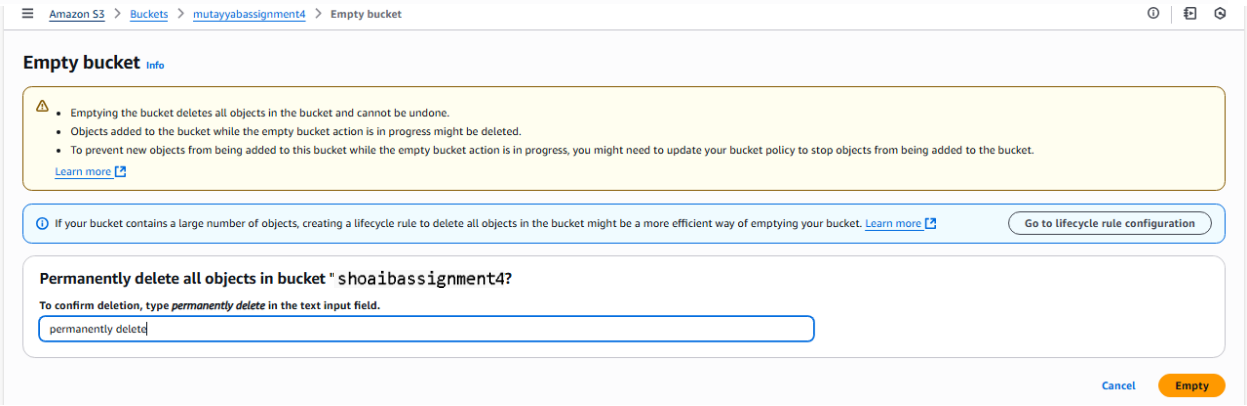
Deleting a bucket to check the **DeleteBucket** Functionality.



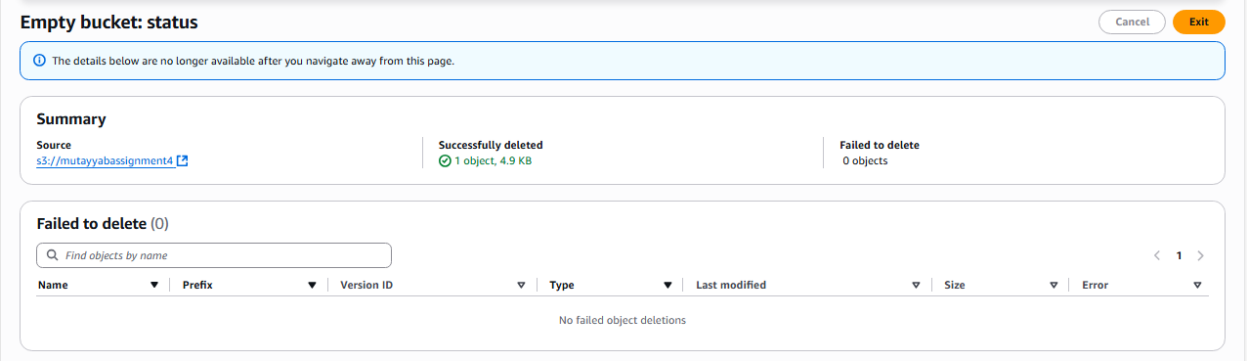
When the bucket has the data, the bucket can't be deleted so we first empty the bucket and then try to delete our bucket.



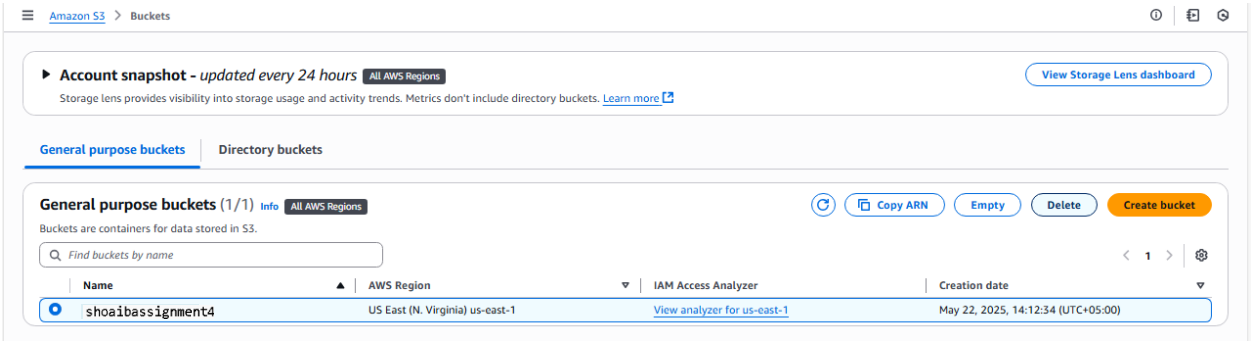
Now Empty the Bucket, Click on Empty bucket Button.

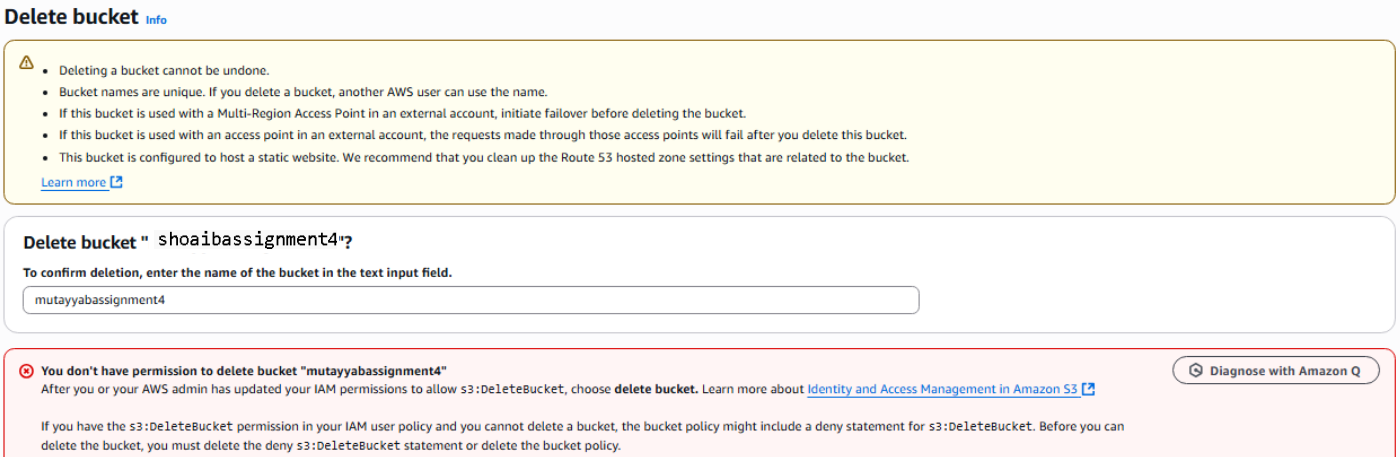


Now the bucket is empty



Now we will be able to delete our bucket to check the **DeleteBucket** policy.





Explanation of how this policy aligns with security best practices:

1. **Principle of Least Privilege**: The policy grants only the minimum necessary permissions (read access) while explicitly denying dangerous actions like delete operations.
2. **Protection Against Accidental Deletion**: By denying DeleteObject and DeleteBucket actions, you prevent accidental or malicious deletion of your website content or the entire bucket.
3. **Prevention of Unauthorized Modifications**: Denying PutObject ensures that no one can modify or overwrite your website files, protecting against defacement or injection attacks.
4. **Public Content Control**: While allowing public read access (necessary for a website), you maintain tight control over what actions are permitted.
5. **Compliance**: This approach helps meet compliance requirements that mandate protection against data loss or unauthorized modifications.
6. **DevOps Security**: In a DevOps pipeline, such policies ensure that production resources can't be accidentally modified or deleted, even if credentials are compromised.
7. **Explicit Deny Overrides**: The explicit deny statements will override any other permissions that might be granted through other policies, providing strong protection.